

Lab 10

GetMaxXY(Irvine32 only):

The GetMaxXY procedure gets the size of the console window's buffer. If the console window buffer is larger than the visible window size, scroll bars appear automatically. GetMaxXY has no input parameters. When it returns, the DX register contains the number of buffer columns and AX contains the number of buffer rows. The possible range of each value can be no greater than 255, which may be smaller than the actual window buffer size.

Sample call:

.data

rows BYTE ?

cols BYTE ?

.code

call GetMaxXY

mov rows,al

mov cols,dl

SetTextColor:

The SetTextColor procedure (Irvine32 library only) sets the foreground and background colors for text output. When calling SetTextColor, assign a color attribute to EAX.

The following predefined color constants can be used for both foreground and background:

black = 0	red = 4	gray = 8	lightRed = 12
blue = 1	magenta = 5	lightBlue = 9	lightMagenta = 13
green = 2	brown = 6	lightGreen = 10	yellow = 14
cyan = 3	lightGray = 7	lightCyan = 11	white = 15

Color constants are defined in the include files named Irvine32.inc and Irvine16.inc. Multiply

the background color by 16 and add it to the foreground color. The following constant, for example, indicates yellow characters on a blue background:

*yellow + (blue * 16)*

The following statements set the color to white on a blue background:

*mov eax,white (blue * 16) ; white on blue*

call SetTextColor

An alternative way to express color constants is to use the SHL operator. You shift the background color leftward by 4 bits before adding it to the foreground color.

yellow + (blue SHL 4)

The bit shifting is performed at assembly time, so it can only be used with constants.

GetTextColor(Irvine32 only):

The GetTextColor procedure gets the current foreground and background colors of the console window. It has no input parameters. It returns the background color in the upper four bits of AL and the foreground color in the lower four bits.

Sample call:

.data

color BYTE ?

.code

call GetTextColor

mov color,AL

Clrscr:

The Clrscr procedure clears the console window. This procedure is typically called at the beginning and end of a program. If you call it at other times, you may need to pause the program by first calling WaitMsg. Doing this allows the user to view information already on the screen before it is erased. Sample call:

call WaitMsg ; "Press any key..."

call Clrscr

Delay:

The Delay procedure pauses the program for a specified number of milliseconds. Before calling Delay, set EAX to the desired interval. Sample call:

mov eax,1000 ; 1 second call Delay

DumpMem:

The DumpMem procedure writes a range of memory to the console window in hexadecimal. Pass it the starting address in ESI, the number of units in ECX, and the unit size in EBX

(1 byte, 2 word, 4 doubleword). The following sample call displays an array of 11 doublewords

in hexadecimal:

.data

```
array DWORD 1,2,3,4,5,6,7,8,9,0Ah,0Bh
```

.code

```
main PROC  
mov esi,OFFSET array ; starting OFFSET  
mov ecx,LENGTHOF array ; number of units  
mov ebx,TYPE array ; doubleword format  
call DumpMem
```

The following output is produced:

```
00000001 00000002 00000003 00000004 00000005 00000006  
00000007 00000008 00000009 0000000A 0000000B
```

DumpRegs:

The DumpRegs procedure displays the EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP, EIP, and EFL (EFLAGS) registers in hexadecimal. It also displays the values of the Carry, Sign, Zero, Overflow, Auxiliary Carry, and Parity flags. Sample call:

```
call DumpRegs
```

Sample output:

```
EAX=00000613 EBX=00000000 ECX=000000FF EDX=00000000  
ESI=00000000 EDI=00000100 EBP=0000091E ESP=000000F6  
EIP=00401026 EFL=00000286 CF=0 SF=1 ZF=0 OF=0 AF=0 PF=1
```

The displayed value of EIP is the offset of the instruction following the call to DumpRegs.

DumpRegs can be useful when debugging programs because it displays a snapshot of the CPU.

It has no input parameters and no return value.

Gotoxy

The Gotoxy procedure locates the cursor at a given row and column in the console window. By default, the console window's X-coordinate range is 0 to 79 and the Y-coordinate range is 0 to 24. When you call Gotoxy, pass the Y-coordinate (row) in DH and the X-coordinate (column)

in DL. Sample call:

```
mov dh,10 ; row 10  
mov dl,20 ; column 20  
call Gotoxy ; locate cursor
```

The user may have resized the console window, so you can call **GetMaxXY** to find out the current number of rows and columns.

IsDigit

The IsDigit procedure determines whether the value in AL is the ASCII code for a valid decimal digit. When calling it, pass an ASCII character in AL. The procedure sets the Zero flag if AL contains a valid decimal digit; otherwise, it clears Zero flag. Sample call:

```
mov AL,somechar
```

```
call IsDigit
```

MsgBox(Irvine32 only):

The MsgBox procedure displays a graphical popup message box with an optional caption. (This works when the program is running in a console window.) Pass it the offset of a string in EDI, which will appear in the inside the box. Optionally, pass the offset of a string for the box's title in EBX. To leave the title blank, set EBX to zero. Sample call:

.data

```
caption db "Dialog Title", 0
```

```
HelloMsg BYTE "This is a pop-up message box.", 0dh,0ah
```

```
BYTE "Click OK to continue...", 0
```

.code

```
mov ebx,OFFSET caption
```

```
mov edi,OFFSET HelloMsg
```

```
call MsgBox
```

Sample output:



MsgBoxAsk(Irvine32 only):

The MsgBoxAsk procedure displays a graphical popup message box with Yes and No buttons. (This works when the program is running in a console window.) Pass it the offset of a question string in EDI, which will appear in the inside the box. Optionally,

pass the offset of a string for the box's title in EBX. To leave the title blank, set EBX to zero.

MsgBoxAsk returns an integer in EAX that tells you which button was selected by the user.

The value will be one of two predefined Windows constants: IDYES (equal to 6) or IDNO (equal to 7). Sample call:

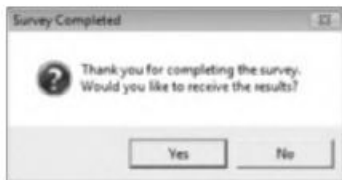
.data

```
caption BYTE "Survey Completed",0
question BYTE "Thank you for completing the survey."
        BYTE 0dh,0ah
        BYTE "Would you like to receive the results?",0
```

.code

```
mov ebx,OFFSET caption
mov edx,OFFSET question
call MsgBoxAsk
;(check return value in EAX)
```

Sample output:



WaitMsg:

The WaitMsg procedure displays the message "Press any key to continue. . ." and waits for the user to press a key. This procedure is useful when you want to pause the screen display before data scrolls off and disappears. It has no input parameters. Sample call:

```
call WaitMsg
```

Tasks:

- Write an assembly program that takes the coordinates of start as in input from user and display star at that coordinate.

- Now building on the above program, take foreground (text color) and background color of start as input from user and color it. Display a color pallet to user for input colors. (Eg Press '1' for red, Press '2' for yellow etc).
- Write an assembly program that takes coordinates and colors from user as an input and write text on that coordinates.
- Write the program that display the grade with respect of the input given by user

Number in Marks	Grade Conversion
90 – 100	A
80 – 89	B
70 – 79	C
60 – 69	D
50 – 59	E
40 – 49	E-
Less than 40	F or Fail

Submission Instructions:

- Write your name, roll no and section on top of your code file
- Do all of your code work in procedures and just call that procedure in main to execute the code. You will make 7 different procedures for each question.

Eg:

```

Q1 proc
;code
ret
Q1 endP

```

- Submit only one .asm file (Format: i22-1234_LAB12.asm)